

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES**FAST SCHOOL OF COMPUTING**

Department of Computer Science

**Data Structures****Lab Manual**

Lab Instructor	Mr. Jawad Khan
Section	BCS-3H
Semester	Fall 2026

Lab 05: Stacks, Queues, and Real-World Applications

1. Learning Objectives

After completing this lab, students will be able to:

1. Understand the Last-In-First-Out (LIFO) principle of a stack and the First-In-First-Out (FIFO) principle of a queue.
2. Implement a stack using an array.
3. Implement a stack using a linked list, without a fixed capacity.
4. Implement a queue using a plain array, and identify the limitation this approach has.
5. Implement a queue using a circular array that correctly reuses freed space.
6. Implement a queue using a linked list.
7. Apply a queue to simulate real-world first-come-first-served systems, including round-robin CPU scheduling.
8. Apply a stack to evaluate arithmetic expressions written in postfix notation.

2. Lab Instructions

- Create a separate .cpp file for each task.
- Use meaningful variable, function, and class names.
- Properly indent and format your code.
- Use meaningful prompts and output labels.
- Add comments where the logic is not immediately obvious.
- Do not use global variables unless explicitly required.
- Do not use STL containers such as stack, queue, vector, or list unless explicitly permitted.
- Dynamically allocated memory must be properly released.
- Make sure there are no memory leaks or dangling pointers.
- Test your program with normal, boundary, and invalid inputs.
- For tasks containing a time-complexity constraint, the required complexity must be followed.

3. Theory and Concept Review

3.1 What Is a Stack?

A stack is a linear data structure in which elements are added and removed at the same end, called the top. This gives a stack Last-In-First-Out (LIFO) behavior: the most recently added element is always the first one removed, in the same way a pile of plates can only be added to or taken from at the top.

A stack supports a small, fixed set of operations, regardless of how it is implemented underneath:

- push: add a new element on top of the stack.
- pop: remove the element currently on top.
- topElement: look at the element currently on top, without removing it.
- isEmpty: report whether the stack currently holds any elements.
- size: report how many elements the stack currently holds.

3.2 Stack Using an Array

A stack can be built on a fixed-size array by keeping a single index that tracks the position of the current top element. Pushing places a value just above the current top, and popping moves the tracking index back down by one, without needing to erase anything. Because the array's size is decided when the stack is created, a push onto a stack that has already reached the array's capacity has to be refused, which is why an array-based stack also needs an isFull operation.

3.3 Stack Using a Linked List

A stack can also be built on a linked list, with each element stored in its own dynamically allocated node. Unlike the array-based version, a linked list stack has no fixed capacity decided in advance, so there is no isFull operation to implement; the stack simply grows one node at a time as needed. Since a stack only ever works at one end, you will need to decide which end of the list to use, and your choice determines whether push and pop can both run in constant time.

3.4 What Is a Queue?

A queue is a linear data structure in which elements are added at one end, called the rear, and removed from the other end, called the front. This gives a queue First-In-First-Out (FIFO) behavior: the first element added is always the first one removed, in the same order a line of people at a counter is served.

A queue supports its own small, fixed set of operations, regardless of how it is implemented underneath:

- enqueue: add a new element at the rear.
- dequeue: remove the element currently at the front.
- frontElement: look at the element currently at the front, without removing it.
- isEmpty: report whether the queue currently holds any elements.
- size: report how many elements the queue currently holds.

3.5 Queue Using a Plain Array

A queue can be built on top of a fixed-size array by keeping two indices: *front*, marking the position of the current front element, and *rear*, marking the next free slot to enqueue into. Enqueuing places a value at *rear* and advances it; dequeuing simply advances *front*, without physically shifting any of the remaining elements.

This approach has a real limitation. Because *front* only ever moves forward and never wraps back around, the slots before the current *front* are freed but never reused. Once *rear* reaches the end of the array, the queue reports itself as full, even if many elements have already been dequeued from the front and most of the array is sitting empty.

3.6 Queue Using a Circular Array

A circular array queue solves this by treating the underlying array as a ring: when *front* or *rear* reaches the last index, the next step wraps back around to index 0 instead of running off the end. This means every freed slot at the front eventually becomes available again for a future enqueue, rather than being permanently lost.

Wrapping around introduces a new question worth thinking through before you start coding: when *front* and *rear* point to the same index, does that mean the queue is empty, or full? Both situations can produce the same index values, so a correct implementation needs some way to tell them apart.

3.7 Queue Using a Linked List

A queue can also be built directly on a linked list, with enqueue inserting a new node at the tail and dequeue removing the node at the head. Unlike either array-based version, a linked list queue has no fixed capacity decided in advance, so there is no *isFull* operation to implement; the queue simply grows one node at a time as needed.

3.8 Stacks and Queues in Real-World Systems

Many everyday systems are naturally modeled as a queue: a line of customers waiting at a counter, print jobs waiting for a printer, or requests waiting to be handled by a server all follow the same first-come-first-served principle a queue provides.

Round-robin scheduling, used by many operating systems to share a single CPU fairly among several processes, is one well-known application of this idea. Each process waiting for the CPU sits in a queue. The process at the front is given a fixed slice of time, called a quantum, to run. If it finishes within that time, it leaves the system; if it still has work left to do, it is placed back at the rear of the queue to wait its turn again. This continues until every process has finished.

Stacks appear wherever the most recently seen item has to be dealt with first. A familiar example is evaluating an arithmetic expression. Calculators and compilers commonly work with expressions in postfix notation, in which every operator is written after the two operands it applies to, so that no parentheses or precedence rules are needed. For example, the postfix expression `3 4 +` means $3 + 4$, and the postfix

expression $5 + 1 + 2 + 4 * + 3 -$ means $5 + ((1 + 2) * 4) - 3$, which equals 14. How a stack helps with evaluating such an expression is for you to work out in Task 8.

4. Lab Tasks

Task 1: Stack Using an Array

Implement a class template named `ArrayStack` that manages a stack of any data type using a fixed-size array, allocated at construction time. Do not use `stack`, `vector`, or any other STL container.

Required Functions

Each row states only what the function must do; designing how it does it is part of the task.

<code>ArrayStack(int capacity);</code>	Design a constructor that allocates an array able to hold up to capacity elements.
<code>void push(T value);</code>	Design a function that adds value on top of the stack.
<code>void pop();</code>	Design a function that removes the element currently on top of the stack.
<code>T topElement();</code>	Design a function that returns the element currently on top, without removing it.
<code>bool isEmpty();</code>	Design a function that returns whether the stack currently holds any elements.
<code>bool isFull();</code>	Design a function that returns whether the stack has reached the array's capacity.
<code>int size();</code>	Design a function that returns the number of elements currently in the stack.
<code>~ArrayStack();</code>	Design a destructor that releases the dynamically allocated array.

Complexity Requirement

- Every function listed above must run in $O(1)$.

Restrictions

- Do not use any STL container.
- Do not shift any elements within the array when pushing or popping.

Required Behaviour

- Calling `push` on a full stack, or `pop` or `topElement` on an empty stack, must not crash the program.

Example

```

ArrayStack<int> s(3);
s.push(10); s.push(20); s.push(30);

isFull()      -> true
topElement()  -> 30

s.push(40);
// refused without crashing, since the stack is full

s.pop();
topElement()  -> 20
size()        -> 2

```

Task 2: Stack Using a Linked List

Implement a class template named `LinkedStack` that manages a stack of any data type using dynamically allocated nodes, with the node implemented as a nested `Node` class, following the same convention used for linked lists in previous labs. Do not use `stack`, `list`, or any other STL container.

Required Node Structure

```

class Node {
public:
    T data;
    Node* next;
};

```

Required Functions

<code>void push(T value);</code>	Design a function that adds a new node containing value on top of the stack.
<code>void pop();</code>	Design a function that removes the node currently on top of the stack.
<code>T topElement();</code>	Design a function that returns the value stored on top, without removing it.
<code>bool isEmpty();</code>	Design a function that returns whether the stack currently holds any elements.
<code>int size();</code>	Design a function that returns the number of elements currently in the stack.
<code>~LinkedStack();</code>	Design a destructor that deletes every remaining node so that no memory is leaked when the stack is destroyed.

Complexity Requirement

- `push`, `pop`, `topElement`, `isEmpty`, and `size` must each run in $O(1)$.

- The destructor must run in $O(n)$.

Restrictions

- Do not use any STL container.
- Neither push nor pop may traverse the stack.

Required Behaviour

- Calling pop or topElement on an empty stack must not crash the program.
- Popping the only remaining element must correctly leave the stack empty, ready to accept further pushes.

Example

```

LinkedStack<int> s;
s.push(1); s.push(2); s.push(3);
s.pop();

topElement()  -> 2
size()        -> 2

```

Task 3: Queue Using a Plain Array

Implement a class template named `ArrayQueue` that manages a queue of any data type using a fixed-size array, allocated at construction time. Do not use `queue`, `vector`, or any other STL container.

Required Functions

Each row states only what the function must do; designing how it does it is part of the task.

<code>ArrayQueue(int capacity);</code>	Design a constructor that allocates an array able to hold up to capacity elements.
<code>void enqueue(T value);</code>	Design a function that adds value at the rear of the queue.
<code>void dequeue();</code>	Design a function that removes the element currently at the front of the queue.
<code>T frontElement();</code>	Design a function that returns the element currently at the front, without removing it.
<code>bool isEmpty();</code>	Design a function that returns whether the queue currently holds any elements.
<code>bool isFull();</code>	Design a function that returns whether the queue has reached the array's capacity, in the sense described in Section 3.5.
<code>int size();</code>	Design a function that returns the number of elements currently in the queue.

<code>~ArrayQueue();</code>	Design a destructor that releases the dynamically allocated array.
-----------------------------	--

Complexity Requirement

- Every function listed above must run in $O(1)$.

Restrictions

- Do not shift any elements within the array when dequeuing; move only the index that tracks the front.
- Do not use any STL container.

Required Behaviour

- Calling enqueue on a full queue, or dequeue or frontElement on an empty queue, must not crash the program.
- As described in Section 3.5, isFull may correctly report true once rear reaches the end of the array, even if elements have already been dequeued from the front and the queue is not logically holding capacity elements. This limitation is expected in this task, and is addressed in Task 4.

Example

```
ArrayQueue<int> q(5);
q.enqueue(1); q.enqueue(2); q.enqueue(3);
q.dequeue(); q.dequeue();
q.enqueue(4); q.enqueue(5);
// size() is now 3: only element 3, 4, and 5 remain

q.enqueue(6);
// isFull() reports true here, since rear has reached the array's
// last index, even though only 3 of the 5 slots are logically in use
```

Task 4: Queue Using a Circular Array

Implement a class template named `CircularArrayQueue` with the same interface as Task 3, but which correctly reuses freed slots at the front of the array, as introduced in Section 3.6. Do not use queue, vector, or any other STL container.

Required Functions

<code>CircularArrayQueue(int capacity);</code>	Design a constructor that allocates an array able to hold up to capacity elements.
<code>void enqueue(T value);</code>	Design a function that adds value at the rear of the queue, wrapping around the array as needed.
<code>void dequeue();</code>	Design a function that removes the element currently at the front of the queue, wrapping around the array as needed.

<code>T frontElement();</code>	Design a function that returns the element currently at the front, without removing it.
<code>bool isEmpty();</code>	Design a function that returns whether the queue currently holds any elements.
<code>bool isFull();</code>	Design a function that returns whether the queue currently holds capacity elements.
<code>int size();</code>	Design a function that returns the number of elements currently in the queue.
<code>~CircularArrayQueue();</code>	Design a destructor that releases the dynamically allocated array.

Complexity Requirement

- Every function listed above must run in $O(1)$.

Restrictions

- Do not shift any elements within the array at any point.
- Do not use any STL container.

Required Behaviour

- `isFull` must correctly report true only when the queue actually holds capacity elements, unlike Task 3.
- Your solution must correctly distinguish between a full queue and an empty queue in every case, including when front and rear refer to the same index. You may use whatever strategy you consider appropriate, but state and justify your choice in a comment.
- Calling `enqueue` on a full queue, or `dequeue` or `frontElement` on an empty queue, must not crash the program.

Example

```
CircularArrayQueue<int> q(3);
q.enqueue(1); q.enqueue(2); q.enqueue(3);
// isFull() is true

q.dequeue();
// isFull() is now false; the freed slot is available again

q.enqueue(4);
// succeeds by wrapping around to reuse the freed slot
// front is 2, and the queue again correctly reports isFull() as true
```

Task 5: Queue Using a Linked List

Implement a class template named `LinkedListQueue` that manages a queue of any data type using dynamically allocated nodes, with the node implemented as a nested `Node` class, following the same convention used for linked lists in previous labs. Do not use `queue`, `list`, or any other STL container.

Required Node Structure

```
class Node {
public:
    T data;
    Node* next;
};
```

Required Functions

<code>void enqueue(T value);</code>	Design a function that adds a new node containing value at the rear of the queue.
<code>void dequeue();</code>	Design a function that removes the node currently at the front of the queue.
<code>T frontElement();</code>	Design a function that returns the value stored at the front, without removing it.
<code>bool isEmpty();</code>	Design a function that returns whether the queue currently holds any elements.
<code>int size();</code>	Design a function that returns the number of elements currently in the queue.
<code>~LinkedList();</code>	Design a destructor that deletes every remaining node so that no memory is leaked when the queue is destroyed.

Complexity Requirement

- enqueue, dequeue, frontElement, and isEmpty must each run in $O(1)$.
- The destructor must run in $O(n)$.

Restrictions

- Do not use any STL container.
- enqueue must not traverse the queue to find the last node; maintain a direct reference to the rear at all times.

Required Behaviour

- Calling dequeue or frontElement on an empty queue must not crash the program.
- Dequeueing the only remaining element must correctly leave the queue empty, ready to accept further enqueues.

Example

```
LinkedList<int> q;
q.enqueue(10); q.enqueue(20); q.enqueue(30);
q.dequeue();

frontElement() -> 20
size()         -> 2
```

Task 6: Ticket Counter Simulation

A single ticket counter serves customers strictly in the order they arrive. You are given each customer's arrival time and the amount of time the counter needs to serve them. Using one of your queue implementations from Tasks 3 through 5, simulate the counter and report, for every customer, the time their service starts, the time it finishes, and how long they waited before being served.

```
void simulateTicketCounter(int arrivalTimes[], int serviceTimes[], int n);
```

Rules of the Simulation

- Customers are given in order of arrival, and this order never changes; the counter cannot serve a later customer before an earlier one.
- If the counter is free when a customer arrives, their service starts immediately at their arrival time.
- If the counter is still busy with an earlier customer when a customer arrives, they must wait until the counter becomes free before their service starts.
- A customer's waiting time is the time their service starts minus their arrival time.

Restrictions

- You must enqueue the customers and process them by dequeuing one at a time; do not simply index into the input arrays as a substitute for using the queue.
- Do not use any STL container.
- Report the average waiting time across all customers, in addition to each customer's individual figures.

Example

```
Arrival times: 0 1 2
Service times: 4 2 1

Customer 1: arrives 0, starts 0, finishes 4, wait 0
Customer 2: arrives 1, starts 4, finishes 6, wait 3
Customer 3: arrives 2, starts 6, finishes 7, wait 4

Average wait: 2.33
```

Task 7: Round-Robin CPU Scheduling Simulation

You are given a list of processes, each with a total burst time representing how much CPU time it needs to finish, and a fixed time quantum. All processes are assumed to arrive at time 0 and are initially queued in the order given. Using one of your queue implementations from Tasks 3 through 5, simulate round-robin scheduling as described in Section 3.8, and report the time at which each process completes.

```
void roundRobin(int processIDs[], int burstTimes[], int n, int quantum);
```

Rules of the Simulation

- The process at the front of the queue runs for either the full quantum, or for the remainder of its burst time, whichever is smaller.
- If a process still has remaining burst time after its turn, it is placed back at the rear of the queue to wait its turn again.
- If a process's remaining burst time reaches zero during its turn, it is finished and is not placed back in the queue.
- The simulation continues until every process has finished.

Restrictions

- Do not use any STL container.
- Do not process the list of processes by simply looping through the input array in order; the queue must actually be used to decide which process runs next, including processes that have been re-queued.

Example

```
Processes: P1 (burst 5), P2 (burst 3), P3 (burst 1)
Quantum: 2

P3 completes at t=5
P2 completes at t=8
P1 completes at t=9
```

Task 8: Postfix Expression Evaluation

A simple calculator receives arithmetic expressions written in postfix notation, as described in Section 3.8, and must compute their value. Using one of your stack implementations from Task 1 or Task 2, write a function that evaluates a postfix expression.

```
bool evaluatePostfix(const string& expression, int& result);
```

Input Format

- The expression is a string of tokens separated by single spaces.
- Each operand is a non-negative integer, and may have more than one digit.
- Each operator is one of + - * /, and division is integer division.
- In an expression such as 8 3 -, the operand that appears first is the left operand, so this evaluates to 8 - 3 = 5.

Complexity Requirement

- Time Complexity: $O(n)$, where n is the number of tokens in the expression.

- Space Complexity: $O(n)$ in the worst case, for the values held by the stack.

Restrictions

- You may use `std::string` for the input expression, but no other STL container.
- You must use one of your own stack implementations to hold intermediate values. If you use `ArrayStack`, choose a capacity large enough for the expressions you test and state it in a comment.
- Clearly state the time and space complexity of your solution in a comment.

Required Behaviour

- The function returns true and stores the value in result only if the expression is valid; otherwise it returns false and leaves result unchanged.
- An expression must be reported as invalid, without crashing the program, if an operator appears when fewer than two values are available, if more than one value remains once every token has been processed, if a division by zero would occur, or if a token is not a number or a recognized operator.

Example

```
Expression: 5 1 2 + 4 * + 3 -  
Result: 14
```

```
Expression: 8 3 -  
Result: 5
```

```
Expression: 2 +  
Invalid (operator without two operands)
```

```
Expression: 4 0 /  
Invalid (division by zero)
```

Submission Guidelines

Submit the following through Google Classroom:

1. Source code (.cpp file) for each completed task, i.e. 25I-XXXX_T1.cpp.
2. Screenshots showing successful execution of each task in a Word file named after your roll number.
3. Answers to the conceptual questions (If any) in the same Word document.
4. Late submissions will be penalized.

Important:

Programs that produce the correct output but violate the specified time-complexity requirement will not receive full credit for that task.